

accurately handle the singularity as $x \rightarrow 0$. The expansions work only for $|v| \leq 1/2$, and so now the recurrence (6.6.5) is used to evaluate f_v at a value $v = \mu$ in this interval. Then one calculates J_μ from

$$J_\mu = \frac{W}{Y'_\mu - Y_\mu f_\mu} \quad (6.6.13)$$

and J'_μ from (6.6.8). The values at the original value of v are determined by scaling as before, and the Y 's are recurred up as before.

Temme's series are

$$Y_v = -\sum_{k=0}^{\infty} c_k g_k \quad Y_{v+1} = -\frac{2}{x} \sum_{k=0}^{\infty} c_k h_k \quad (6.6.14)$$

Here

$$c_k = \frac{(-x^2/4)^k}{k!} \quad (6.6.15)$$

while the coefficients g_k and h_k are defined in terms of quantities p_k , q_k , and f_k that can be found by recursion:

$$\begin{aligned} g_k &= f_k + \frac{2}{v} \sin^2\left(\frac{v\pi}{2}\right) q_k \\ h_k &= -k g_k + p_k \\ p_k &= \frac{p_{k-1}}{k-v} \\ q_k &= \frac{q_{k-1}}{k+v} \\ f_k &= \frac{k f_{k-1} + p_{k-1} + q_{k-1}}{k^2 - v^2} \end{aligned} \quad (6.6.16)$$

The initial values for the recurrences are

$$\begin{aligned} p_0 &= \frac{1}{\pi} \left(\frac{x}{2}\right)^{-v} \Gamma(1+v) \\ q_0 &= \frac{1}{\pi} \left(\frac{x}{2}\right)^v \Gamma(1-v) \\ f_0 &= \frac{2}{\pi} \frac{v\pi}{\sin v\pi} \left[\cosh \sigma \Gamma_1(v) + \frac{\sinh \sigma}{\sigma} \ln\left(\frac{2}{x}\right) \Gamma_2(v) \right] \end{aligned} \quad (6.6.17)$$

with

$$\begin{aligned} \sigma &= v \ln\left(\frac{2}{x}\right) \\ \Gamma_1(v) &= \frac{1}{2v} \left[\frac{1}{\Gamma(1-v)} - \frac{1}{\Gamma(1+v)} \right] \\ \Gamma_2(v) &= \frac{1}{2} \left[\frac{1}{\Gamma(1-v)} + \frac{1}{\Gamma(1+v)} \right] \end{aligned} \quad (6.6.18)$$

The whole point of writing the formulas in this way is that the potential problems as $v \rightarrow 0$ can be controlled by evaluating $v\pi/\sin v\pi$, $\sinh \sigma/\sigma$, and Γ_1 carefully. In particular, Temme gives Chebyshev expansions for $\Gamma_1(v)$ and $\Gamma_2(v)$. We have rearranged his expansion for Γ_1 to be explicitly an even series in v for more efficient evaluation, as explained in §5.8.

Because J_v , Y_v , J'_v , and Y'_v are all calculated simultaneously, a single void function sets them all. You then grab those that you need directly from the object. Alternatively, the functions `jnu` and `ynu` can be used. (We've omitted similar helper functions for the derivatives, but you can easily add them.) The object `Bessel` contains various other methods that will be discussed below.

The routines assume $\nu \geq 0$. For negative ν you can use the reflection formulas

$$\begin{aligned} J_{-\nu} &= \cos \nu \pi J_{\nu} - \sin \nu \pi Y_{\nu} \\ Y_{-\nu} &= \sin \nu \pi J_{\nu} + \cos \nu \pi Y_{\nu} \end{aligned} \quad (6.6.19)$$

The routine also assumes $x > 0$. For $x < 0$, the functions are in general complex but expressible in terms of functions with $x > 0$. For $x = 0$, Y_{ν} is singular. The complex arithmetic is carried out explicitly with real variables.

besselfrac.h

```
struct Bessel {
Object for Bessel functions of arbitrary order  $\nu$ , and related functions.
    static const Int NUSE1=7, NUSE2=8;
    static const Doub c1[NUSE1],c2[NUSE2];
    Doub xo,nuo;                                Saved  $x$  and  $\nu$  from last call.
    Doub jo,yo,jpo,ypo;                          Set by besseljy.
    Doub io,ko,ipo,kpo;                          Set by besselik.
    Doub aio,bio,aipo,bipo;                      Set by airy.
    Doub sphjo,sphyo,sphjpo,sphypo;             Set by sphbes.
    Int sphno;

    Bessel() : xo(9.99e99), nuo(9.99e99), sphno(-9999) {}
    Default constructor. No arguments.

    void besseljy(const Doub nu, const Doub x);
    Calculate Bessel functions  $J_{\nu}(x)$  and  $Y_{\nu}(x)$  and their derivatives.
    void besselik(const Doub nu, const Doub x);
    Calculate Bessel functions  $I_{\nu}(x)$  and  $K_{\nu}(x)$  and their derivatives.

    Doub jnu(const Doub nu, const Doub x) {
    Simple interface returning  $J_{\nu}(x)$ .
        if (nu != nuo || x != xo) besseljy(nu,x);
        return jo;
    }
    Doub ynu(const Doub nu, const Doub x) {
    Simple interface returning  $Y_{\nu}(x)$ .
        if (nu != nuo || x != xo) besseljy(nu,x);
        return yo;
    }
    Doub inu(const Doub nu, const Doub x) {
    Simple interface returning  $I_{\nu}(x)$ .
        if (nu != nuo || x != xo) besselik(nu,x);
        return io;
    }
    Doub knu(const Doub nu, const Doub x) {
    Simple interface returning  $K_{\nu}(x)$ .
        if (nu != nuo || x != xo) besselik(nu,x);
        return ko;
    }

    void airy(const Doub x);
    Calculate Airy functions  $Ai(x)$  and  $Bi(x)$  and their derivatives.
    Doub airy_ai(const Doub x);
    Simple interface returning  $Ai(x)$ .
    Doub airy_bi(const Doub x);
    Simple interface returning  $Bi(x)$ .

    void sphbes(const Int n, const Doub x);
    Calculate spherical Bessel functions  $j_n(x)$  and  $y_n(x)$  and their derivatives.
    Doub sphbesj(const Int n, const Doub x);
    Simple interface returning  $j_n(x)$ .
    Doub sphbesy(const Int n, const Doub x);
    Simple interface returning  $y_n(x)$ .
```

```

inline Doub chebev(const Doub *c, const Int m, const Doub x) {
Utility used by besseljy and besselik, evaluates Chebyshev series.
    Doub d=0.0,dd=0.0,sv;
    Int j;
    for (j=m-1;j>0;j--) {
        sv=d;
        d=2.*x*d-dd+c[j];
        dd=sv;
    }
    return x*d-dd+0.5*c[0];
}
};

const Doub Bessel::c1[7] = {-1.142022680371168e0,6.5165112670737e-3,
    3.087090173086e-4,-3.4706269649e-6,6.9437664e-9,3.67795e-11,
    -1.356e-13};
const Doub Bessel::c2[8] = {1.843740587300905e0,-7.68528408447867e-2,
    1.2719271366546e-3,-4.9717367042e-6,-3.31261198e-8,2.423096e-10,
    -1.702e-13,-1.49e-15};

```

The code listing for `Bessel::besseljy` is in a Webnote [4].

6.6.2 Modified Bessel Functions

Steed's method does not work for modified Bessel functions because in this case CF2 is purely imaginary and we have only three relations among the four functions. Temme [3] has given a normalization condition that provides the fourth relation.

The Wronskian relation is

$$W \equiv I_\nu K'_\nu - K_\nu I'_\nu = -\frac{1}{x} \quad (6.6.20)$$

The continued fraction CF1 becomes

$$f_\nu \equiv \frac{I'_\nu}{I_\nu} = \frac{\nu}{x} + \frac{1}{2(\nu+1)/x + \frac{1}{2(\nu+2)/x + \dots}} \quad (6.6.21)$$

To get CF2 and the normalization condition in a convenient form, consider the sequence of confluent hypergeometric functions

$$z_n(x) = U(\nu + 1/2 + n, 2\nu + 1, 2x) \quad (6.6.22)$$

for fixed ν . Then

$$K_\nu(x) = \pi^{1/2} (2x)^\nu e^{-x} z_0(x) \quad (6.6.23)$$

$$\frac{K_{\nu+1}(x)}{K_\nu(x)} = \frac{1}{x} \left[\nu + \frac{1}{2} + x + \left(\nu^2 - \frac{1}{4} \right) \frac{z_1}{z_0} \right] \quad (6.6.24)$$

Equation (6.6.23) is the standard expression for K_ν in terms of a confluent hypergeometric function, while equation (6.6.24) follows from relations between contiguous confluent hypergeometric functions (equations 13.4.16 and 13.4.18 in Ref. [5]). Now the functions z_n satisfy the three-term recurrence relation (equation 13.4.15 in Ref. [5])

$$z_{n-1}(x) = b_n z_n(x) + a_{n+1} z_{n+1} \quad (6.6.25)$$

with

$$\begin{aligned} b_n &= 2(n+x) \\ a_{n+1} &= -[(n+1/2)^2 - \nu^2] \end{aligned} \quad (6.6.26)$$

Following the steps leading to equation (5.4.18), we get the continued fraction CF2

$$\frac{z_1}{z_0} = \frac{1}{b_1 + \frac{a_2}{b_2 + \dots}} \quad (6.6.27)$$

from which (6.6.24) gives $K_{\nu+1}/K_{\nu}$ and thus K'_{ν}/K_{ν} .

Temme's normalization condition is that

$$\sum_{n=0}^{\infty} C_n z_n = \left(\frac{1}{2x}\right)^{\nu+1/2} \quad (6.6.28)$$

where

$$C_n = \frac{(-1)^n}{n!} \frac{\Gamma(\nu + 1/2 + n)}{\Gamma(\nu + 1/2 - n)} \quad (6.6.29)$$

Note that the C_n 's can be determined by recursion:

$$C_0 = 1, \quad C_{n+1} = -\frac{a_{n+1}}{n+1} C_n \quad (6.6.30)$$

We use the condition (6.6.28) by finding

$$S = \sum_{n=1}^{\infty} C_n \frac{z_n}{z_0} \quad (6.6.31)$$

Then

$$z_0 = \left(\frac{1}{2x}\right)^{\nu+1/2} \frac{1}{1+S} \quad (6.6.32)$$

and (6.6.23) gives K_{ν} .

Thompson and Barnett [6] have given a clever method of doing the sum (6.6.31) simultaneously with the forward evaluation of the continued fraction CF2. Suppose the continued fraction is being evaluated as

$$\frac{z_1}{z_0} = \sum_{n=0}^{\infty} \Delta h_n \quad (6.6.33)$$

where the increments Δh_n are being found by, e.g., Steed's algorithm or the modified Lentz's algorithm of §5.2. Then the approximation to S keeping the first N terms can be found as

$$S_N = \sum_{n=1}^N Q_n \Delta h_n \quad (6.6.34)$$

Here

$$Q_n = \sum_{k=1}^n C_k q_k \quad (6.6.35)$$

and q_k is found by recursion from

$$q_{k+1} = (q_{k-1} - b_k q_k)/a_{k+1} \quad (6.6.36)$$

starting with $q_0 = 0, q_1 = 1$. For the case at hand, approximately three times as many terms are needed to get S to converge as are needed simply for CF2 to converge.

To find K_{ν} and $K_{\nu+1}$ for small x we use series analogous to (6.6.14):

$$K_{\nu} = \sum_{k=0}^{\infty} c_k f_k \quad K_{\nu+1} = \frac{2}{x} \sum_{k=0}^{\infty} c_k h_k \quad (6.6.37)$$

Here

$$\begin{aligned}
 c_k &= \frac{(x^2/4)^k}{k!} \\
 h_k &= -kf_k + p_k \\
 p_k &= \frac{pk-1}{k-v} \\
 q_k &= \frac{qk-1}{k+v} \\
 f_k &= \frac{kf_{k-1} + p_{k-1} + q_{k-1}}{k^2 - v^2}
 \end{aligned} \tag{6.6.38}$$

The initial values for the recurrences are

$$\begin{aligned}
 p_0 &= \frac{1}{2} \left(\frac{x}{2}\right)^{-v} \Gamma(1+v) \\
 q_0 &= \frac{1}{2} \left(\frac{x}{2}\right)^v \Gamma(1-v) \\
 f_0 &= \frac{v\pi}{\sin v\pi} \left[\cosh \sigma \Gamma_1(v) + \frac{\sinh \sigma}{\sigma} \ln \left(\frac{2}{x}\right) \Gamma_2(v) \right]
 \end{aligned} \tag{6.6.39}$$

Both the series for small x , and CF2 and the normalization relation (6.6.28) require $|v| \leq 1/2$. In both cases, therefore, we recurse I_v down to a value $v = \mu$ in this interval, find K_μ there, and recurse K_v back up to the original value of v .

The routine assumes $v \geq 0$. For negative v use the reflection formulas

$$\begin{aligned}
 I_{-v} &= I_v + \frac{2}{\pi} \sin(v\pi) K_v \\
 K_{-v} &= K_v
 \end{aligned} \tag{6.6.40}$$

Note that for large x , $I_v \sim e^x$ and $K_v \sim e^{-x}$, and so these functions will overflow or underflow. It is often desirable to be able to compute the scaled quantities $e^{-x} I_v$ and $e^x K_v$. Simply omitting the factor e^{-x} in equation (6.6.23) will ensure that all four quantities will have the appropriate scaling. If you also want to scale the four quantities for small x when the series in equation (6.6.37) are used, you must multiply each series by e^x .

As with `besseljy`, you can either call the void function `besselik`, and then retrieve the function and/or derivative values from the object, or else just call `inu` or `knu`.

The code listing for `Bessel::besselik` is in a Webnote [4].

6.6.3 Airy Functions

For positive x , the Airy functions are defined by

$$\text{Ai}(x) = \frac{1}{\pi} \sqrt{\frac{x}{3}} K_{1/3}(z) \tag{6.6.41}$$

$$\text{Bi}(x) = \sqrt{\frac{x}{3}} [I_{1/3}(z) + I_{-1/3}(z)] \tag{6.6.42}$$

where

$$z = \frac{2}{3} x^{3/2} \tag{6.6.43}$$

By using the reflection formula (6.6.40), we can convert (6.6.42) into the computationally more useful form

$$\text{Bi}(x) = \sqrt{x} \left[\frac{2}{\sqrt{3}} I_{1/3}(z) + \frac{1}{\pi} K_{1/3}(z) \right] \tag{6.6.44}$$

so that `Ai` and `Bi` can be evaluated with a single call to `besselik`.

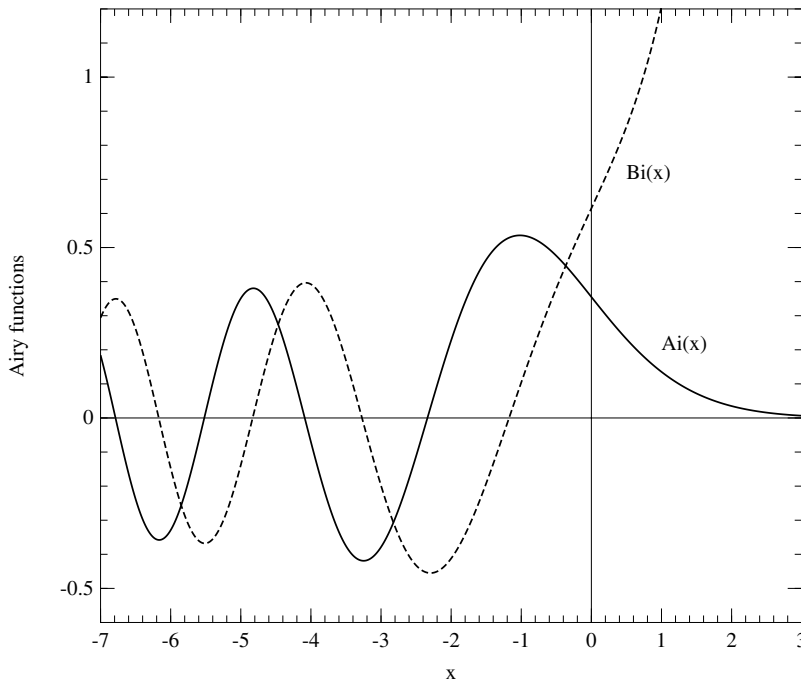


Figure 6.6.1. Airy functions $\text{Ai}(x)$ and $\text{Bi}(x)$.

The derivatives should not be evaluated by simply differentiating the above expressions because of possible subtraction errors near $x = 0$. Instead, use the equivalent expressions

$$\begin{aligned}\text{Ai}'(x) &= -\frac{x}{\pi\sqrt{3}}K_{2/3}(z) \\ \text{Bi}'(x) &= x \left[\frac{2}{\sqrt{3}}I_{2/3}(z) + \frac{1}{\pi}K_{2/3}(z) \right]\end{aligned}\tag{6.6.45}$$

The corresponding formulas for negative arguments are

$$\begin{aligned}\text{Ai}(-x) &= \frac{\sqrt{x}}{2} \left[J_{1/3}(z) - \frac{1}{\sqrt{3}}Y_{1/3}(z) \right] \\ \text{Bi}(-x) &= -\frac{\sqrt{x}}{2} \left[\frac{1}{\sqrt{3}}J_{1/3}(z) + Y_{1/3}(z) \right] \\ \text{Ai}'(-x) &= \frac{x}{2} \left[J_{2/3}(z) + \frac{1}{\sqrt{3}}Y_{2/3}(z) \right] \\ \text{Bi}'(-x) &= \frac{x}{2} \left[\frac{1}{\sqrt{3}}J_{2/3}(z) - Y_{2/3}(z) \right]\end{aligned}\tag{6.6.46}$$

besselfrac.h

```
void Bessel::airy(const Doub x) {
```

Sets aio, bio, aipo, and bipo, respectively, to the Airy functions $\text{Ai}(x)$, $\text{Bi}(x)$ and their derivatives $\text{Ai}'(x)$, $\text{Bi}'(x)$.

```
    static const Doub PI=3.141592653589793238,
        ONOVRT=0.577350269189626, THR=1./3., TWOTHR=2.*THR;
    Doub absx, rootx, z;
    absx=abs(x);
    rootx=sqrt(absx);
    z=TWOTHR*absx*rootx;
```

```

if (x > 0.0) {
    besselik(THR,z);
    aio = rootx*ONOVRT*ko/PI;
    bio = rootx*(ko/PI+2.0*ONOVRT*io);
    besselik(TWOTHR,z);
    aipo = -x*ONOVRT*ko/PI;
    bipo = x*(ko/PI+2.0*ONOVRT*io);
} else if (x < 0.0) {
    besseljy(THR,z);
    aio = 0.5*rootx*(jo-ONOVRT*yo);
    bio = -0.5*rootx*(yo+ONOVRT*jo);
    besseljy(TWOTHR,z);
    aipo = 0.5*absx*(ONOVRT*yo+jo);
    bipo = 0.5*absx*(ONOVRT*jo-yo);
} else {
    Case x = 0.
    aio=0.355028053887817;
    bio=aio/ONOVRT;
    aipo = -0.258819403792807;
    bipo = -aipo/ONOVRT;
}
}

Doub Bessel::airy_ai(const Doub x) {
Simple interface returning Ai(x).
    if (x != xo) airy(x);
    return aio;
}
Doub Bessel::airy_bi(const Doub x) {
Simple interface returning Bi(x).
    if (x != xo) airy(x);
    return bio;
}

```

6.6.4 Spherical Bessel Functions

For integer n , spherical Bessel functions are defined by

$$\begin{aligned}
 j_n(x) &= \sqrt{\frac{\pi}{2x}} J_{n+\frac{1}{2}}(x) \\
 y_n(x) &= \sqrt{\frac{\pi}{2x}} Y_{n+\frac{1}{2}}(x)
 \end{aligned}
 \tag{6.6.47}$$

They can be evaluated by a call to `besseljy`, and the derivatives can safely be found from the derivatives of equation (6.6.47).

Note that in the continued fraction CF2 in (6.6.3) just the first term survives for $\nu = 1/2$. Thus one can make a very simple algorithm for spherical Bessel functions along the lines of `besseljy` by always recursing j_n down to $n = 0$, setting p and q from the first term in CF2, and then recursing y_n up. No special series is required near $x = 0$. However, `besseljy` is already so efficient that we have not bothered to provide an independent routine for spherical Bessels.

```

void Bessel::sphbes(const Int n, const Doub x) {
Sets sphjo, sphyo, sphjpo, and sphypo, respectively, to the spherical Bessel functions  $j_n(x)$ ,  $y_n(x)$ , and their derivatives  $j'_n(x)$ ,  $y'_n(x)$  for integer  $n$  (which is saved as sphno).
    const Doub RTPIO2=1.253314137315500251;
    Doub factor,order;
    if (n < 0 || x <= 0.0) throw("bad arguments in sphbes");
    order=n+0.5;
    besseljy(order,x);
    factor=RTPIO2/sqrt(x);
    sphjo=factor*jo;

```

`besselfrac.h`

```

    sphyo=factor*yo;
    sphjpo=factor*jpo-sphjo/(2.*x);
    sphypo=factor*ypo-sphyo/(2.*x);
    sphno = n;
}

Doub Bessel::sphbesj(const Int n, const Doub x) {
Simple interface returning  $j_n(x)$ .
    if (n != sphno || x != xo) sphbes(n,x);
    return sphjo;
}

Doub Bessel::sphbesy(const Int n, const Doub x) {
Simple interface returning  $y_n(x)$ .
    if (n != sphno || x != xo) sphbes(n,x);
    return sphyo;
}

```

CITED REFERENCES AND FURTHER READING:

- Barnett, A.R., Feng, D.H., Steed, J.W., and Goldfarb, L.J.B. 1974, "Coulomb Wave Functions for All Real η and ρ ," *Computer Physics Communications*, vol. 8, pp. 377–395.[1]
- Temme, N.M. 1976, "On the Numerical Evaluation of the Ordinary Bessel Function of the Second Kind," *Journal of Computational Physics*, vol. 21, pp. 343–350[2]; 1975, *op. cit.*, vol. 19, pp. 324–337.[3]
- Numerical Recipes Software 2007, "Bessel Function Implementations," *Numerical Recipes Web-note No. 8*, at <http://www.nr.com/webnotes?8> [4]
- Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at <http://www.nr.com/aands>, Chapter 10.[5]
- Thompson, I.J., and Barnett, A.R. 1987, "Modified Bessel Functions $I_\nu(z)$ and $K_\nu(z)$ of Real Order and Complex Argument, to Selected Accuracy," *Computer Physics Communications*, vol. 47, pp. 245–257.[6]
- Barnett, A.R. 1981, "An Algorithm for Regular and Irregular Coulomb and Bessel functions of Real Order to Machine Accuracy," *Computer Physics Communications*, vol. 21, pp. 297–314.
- Thompson, I.J., and Barnett, A.R. 1986, "Coulomb and Bessel Functions of Complex Arguments and Order," *Journal of Computational Physics*, vol. 64, pp. 490–509.

6.7 Spherical Harmonics

Spherical harmonics occur in a large variety of physical problems, for example, whenever a wave equation, or Laplace's equation, is solved by separation of variables in spherical coordinates. The spherical harmonic $Y_{lm}(\theta, \phi)$, $-l \leq m \leq l$, is a function of the two coordinates θ, ϕ on the surface of a sphere.

The spherical harmonics are orthogonal for different l and m , and they are normalized so that their integrated square over the sphere is unity:

$$\int_0^{2\pi} d\phi \int_{-1}^1 d(\cos \theta) Y_{l'm'}^*(\theta, \phi) Y_{lm}(\theta, \phi) = \delta_{l'l} \delta_{m'm} \quad (6.7.1)$$

Here the asterisk denotes complex conjugation.

Mathematically, the spherical harmonics are related to *associated Legendre polynomials* by the equation

$$Y_{lm}(\theta, \phi) = \sqrt{\frac{2l+1}{4\pi} \frac{(l-m)!}{(l+m)!}} P_l^m(\cos \theta) e^{im\phi} \quad (6.7.2)$$

By using the relation

$$Y_{l,-m}(\theta, \phi) = (-1)^m Y_{lm}^*(\theta, \phi) \quad (6.7.3)$$

we can always relate a spherical harmonic to an associated Legendre polynomial with $m \geq 0$. With $x \equiv \cos \theta$, these are defined in terms of the ordinary Legendre polynomials (cf. §4.6 and §5.4) by

$$P_l^m(x) = (-1)^m (1-x^2)^{m/2} \frac{d^m}{dx^m} P_l(x) \quad (6.7.4)$$

Be careful: There are alternative normalizations for the associated Legendre polynomials and alternative sign conventions.

The first few associated Legendre polynomials, and their corresponding normalized spherical harmonics, are

$P_0^0(x) = 1$	$Y_{00} = \sqrt{\frac{1}{4\pi}}$	(6.7.5)
$P_1^1(x) = -(1-x^2)^{1/2}$	$Y_{11} = -\sqrt{\frac{3}{8\pi}} \sin \theta e^{i\phi}$	
$P_1^0(x) = x$	$Y_{10} = \sqrt{\frac{3}{4\pi}} \cos \theta$	
$P_2^2(x) = 3(1-x^2)$	$Y_{22} = \frac{1}{4} \sqrt{\frac{15}{2\pi}} \sin^2 \theta e^{2i\phi}$	
$P_2^1(x) = -3(1-x^2)^{1/2}x$	$Y_{21} = -\sqrt{\frac{15}{8\pi}} \sin \theta \cos \theta e^{i\phi}$	
$P_2^0(x) = \frac{1}{2}(3x^2 - 1)$	$Y_{20} = \sqrt{\frac{5}{4\pi}} \left(\frac{3}{2} \cos^2 \theta - \frac{1}{2} \right)$	

There are many bad ways to evaluate associated Legendre polynomials numerically. For example, there are explicit expressions, such as

$$P_l^m(x) = \frac{(-1)^m (l+m)!}{2^m m! (l-m)!} (1-x^2)^{m/2} \left[1 - \frac{(l-m)(m+l+1)}{1!(m+1)} \left(\frac{1-x}{2} \right) + \frac{(l-m)(l-m-1)(m+l+1)(m+l+2)}{2!(m+1)(m+2)} \left(\frac{1-x}{2} \right)^2 - \dots \right] \quad (6.7.6)$$

where the polynomial continues up through the term in $(1-x)^{l-m}$. (See [1] for this and related formulas.) This is not a satisfactory method because evaluation of the polynomial involves delicate cancellations between successive terms, which alternate in sign. For large l , the individual terms in the polynomial become very much larger than their sum, and all accuracy is lost.

In practice, (6.7.6) can be used only in single precision (32-bit) for l up to 6 or 8, and in double precision (64-bit) for l up to 15 or 18, depending on the precision required for the answer. A more robust computational procedure is therefore desirable, as follows.